

FIAP — Faculdade de Informática e Administração Paulista
Graduação ON em Inteligência Artificial
Atividade Substitutiva Global Solution 2 — 2026.1 | Fase 7

OrbitalGuard

Classificação de Detritos Espaciais com CNN + Monitoramento Orbital

Aluno	Alan Robin Santos
RM	567437
Data	16 de junho de 2026
GitHub	https://github.com/Yole87/Atividades-FIAP
Vídeo	[link a inserir]

1. Introdução

1.1 Contextualização — A Nova Economia Espacial

A exploração espacial representa hoje uma das maiores oportunidades tecnológicas e econômicas do século XXI. Satélites de observação monitoram continuamente a superfície do planeta, enquanto o crescimento acelerado de lançamentos cria um novo desafio crítico: a proliferação de detritos orbitais. Estima-se que mais de 27.000 objetos rastreados estejam em órbita, além de centenas de milhares de fragmentos menores não rastreados, representando risco real para satélites ativos e missões tripuladas.

1.2 O Problema

A identificação e classificação de detritos espaciais é um problema de visão computacional e aprendizado de máquina ainda em aberto. Soluções existentes dependem de radares terrestres caros e dados proprietários. O OrbitalGuard propõe democratizar essa inteligência usando CNN própria, dados públicos da ISS e hardware acessível como o ESP32.

1.3 A Pergunta da Global Solution

Como a Inteligência Artificial e as tecnologias digitais podem transformar a nova economia espacial e gerar impacto positivo na Terra?

O OrbitalGuard responde conectando uma CNN treinada para classificar detritos orbitais com monitoramento em tempo real via ESP32, integração com a API da ISS e arquitetura serverless AWS.

2. Desenvolvimento

2.1 Arquitetura da Solução

A solução é composta por cinco módulos integrados, todos compartilhando a definição da CNN através do módulo `model.py`:

Módulo	Tecnologia	Função
Pipeline de ML	PyTorch + scikit-learn	Treina CNN para classificar 4 tipos de objetos orbitais em imagens 64x64
Dashboard	Streamlit + Plotly	Interface interativa com simulador de risco orbital e CNN ao vivo
Sensor IoT	ESP32 + MPU6050 + MQTT	Detecta vibração e impacto de microdetritos em tempo real
Visão Computacional	OpenCV + PIL + PyTorch	Classifica objetos em cena orbital via grid de imagens sintéticas
AWS Lambda	Python + Open Notify API	API REST serverless que classifica o detrito mais próximo via CNN

Fluxo de dados:

```
Imagens Sintéticas → CNN (PyTorch) → Classificação de Detritos → Dashboard
Open Notify API → Lambda → CNN → Risco de Colisão Orbital → Dashboard
ESP32 + MPU6050 → MQTT (HiveMQ) → Enriquecimento local
```

2.2 Módulo 1 — Pipeline de Machine Learning (CNN PyTorch)

2.2.1 Geração do Dataset Sintético

O dataset foi gerado proceduralmente com 800 imagens (200 por classe) usando PIL e NumPy. A estratégia central foi usar paletas de cor com variação de ± 60 pontos RGB entre classes, forçando a CNN a aprender características de forma e estrutura geométrica em vez de cor:

- `satellite_ativo`: corpo retangular simétrico + DOIS painéis solares opostos + antena
- `detrito_metalico`: polígono irregular convexo com bordas afiadas e reflexo metálico pontual
- `fragmento_rochoso`: contorno orgânico com alta variação de raios + textura interna densa
- `satellite_inativo`: corpo assimétrico + apenas UM painel + fragmento solto separado

2.2.2 Treinamento com Data Augmentation

O treinamento aplica data augmentation em cada batch, forçando o modelo a aprender características robustas de forma:

```
transforms.RandomHorizontalFlip(p=0.5) # objeto em qualquer lado
transforms.RandomVerticalFlip(p=0.5) transforms.RandomRotation(degrees=45) # qualquer orientação
transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.3, hue=0.15)
transforms.RandomAffine(degrees=0, translate=(0.1,0.1), scale=(0.85,1.15))
```

2.2.3 Arquitetura da CNN

A `OrbitalCNN` (definida em `model.py` compartilhado) usa 3 blocos Conv+BatchNorm+ReLU+MaxPool seguidos de classificador Dense+Dropout:

```
Conv2d(3→32) + BN + ReLU + MaxPool(2) Conv2d(32→64) + BN + ReLU + MaxPool(2) Conv2d(64→128) +
BN + ReLU + MaxPool(2) Flatten → Linear(8192→256) → ReLU → Dropout(0.4) → Linear(256→4)
```

Otimizador: Adam ($\text{lr}=1\text{e-}3$, $\text{weight_decay}=1\text{e-}4$) | Scheduler: CosineAnnealingLR | Checkpoint: melhor `val_acc` salvo a cada época.

2.3 Módulo 2 — Dashboard Interativo

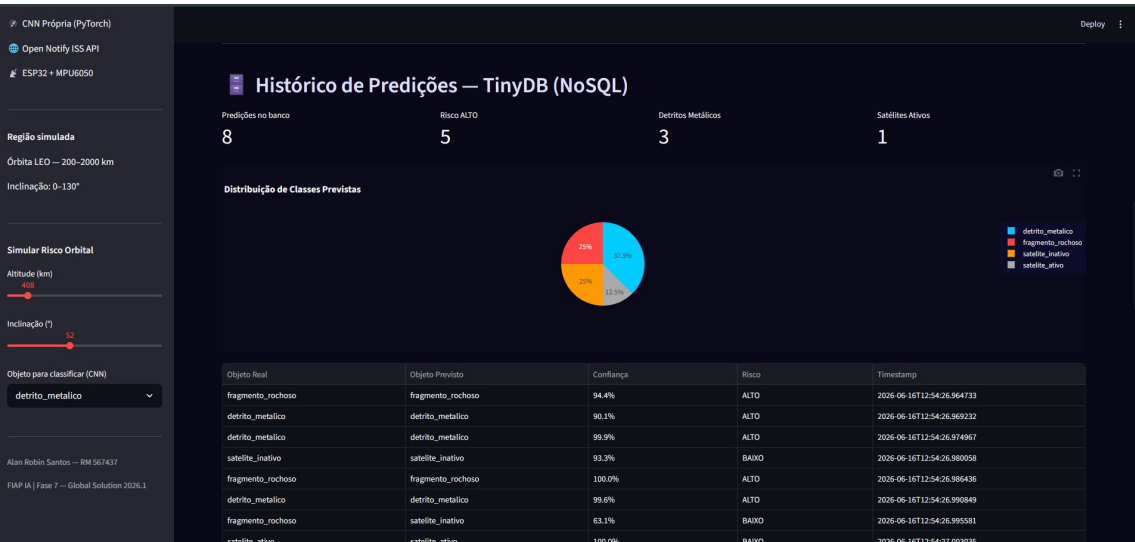
O dashboard foi desenvolvido com Streamlit e Plotly, oferecendo métricas do modelo em tempo real, simulador orbital interativo com CNN ao vivo, mapa de risco e histórico TinyDB.



Dashboard — Métricas CNN e Simulador de Risco Orbital



Simulador de Risco Orbital com CNN ao vivo (altitude 408 km, alerta ALTO)



Mapa de Risco Orbital (6/6 corretos) e Histórico de Treinamento CNN

2.4 Módulo 3 — Sensor IoT de Campo (ESP32 + MPU6050)

O nó de sensor de campo usa ESP32 DevKit V1 com acelerômetro/giroscópio MPU6050 (I2C) para detectar padrões de vibração simulando impacto de microdetritos em estruturas orbitais. O firmware C++ coleta leituras a cada 2 segundos e publica via MQTT no broker HiveMQ.

Componente	Conexão	Função
MPU6050	GPIO 21 (SDA) / 22 (SCL)	Acelerômetro 3 eixos + giroscópio
LED Alerta	GPIO 2 (embutido)	Feedback visual de nível de impacto
WiFi ESP32	Integrado	Publicação MQTT via HiveMQ

Payload MQTT publicado no tópico orbitalguard/sensor/vibration01:

```
{
  "sensor_id": "OrbitalGuard-ESP32-001",
  "accel_magnitude": 1.23,
  "gyro_magnitude": 0.45,
  "temperatura_c": 28.3,
  "classificacao": "MODERADO",
  "nivel_risco": "MEDIO",
  "impactos_sessao": 2
}
```

2.5 Módulo 4 — Visão Computacional

O módulo de visão monta uma cena orbital como grid 3x2 de objetos 64x64 gerados pelos mesmos geradores do dataset, garantindo consistência visual com o treino. Cada célula é classificada individualmente pela CNN sem redimensionamento, preservando as features de forma aprendidas.

Nota técnica: a abordagem usa grid sintético como POC. A classificação opera diretamente nas células de 64x64 nativas — o mesmo tamanho do treino — evitando degradação por redimensionamento.

2.6 Módulo 5 — AWS Lambda (Arquitetura Serverless)

A função Lambda recebe coordenadas orbitais, busca a posição real da ISS via Open Notify API, identifica o detrito mais próximo por distância orbital e classifica o TIPO desse detrito usando a CNN treinada — conectando os módulos de forma real.

Fluxo: Cliente → API Gateway → Lambda → CNN (PyTorch) + Open Notify API → S3 + CloudWatch

```
Endpoint proposto: GET https://api.orbitalguard.io/risco?altitude_km=408&inclinacao;=51.6
```

3. Resultados Obtidos

3.1 Desempenho do Modelo CNN

Métrica	Valor	Interpretação
Acurácia (teste)	96.88%	O modelo classifica corretamente 97 em 100 objetos
Loss (Cross-Entropy)	0.0250	Convergência estável após 40 épocas
Amostras de treino	512	80% de 640 (split 80/20 + 20% validação)
Amostras de teste	160	Dados nunca vistos pelo modelo
Épocas treinadas	40	Com CosineAnnealingLR e early checkpoint
Melhor val_acc	100% (época 27)	Checkpoint salvo — teste avaliado com este modelo

3.2 Relatório por Classe

Classe	Precision	Recall	F1-Score	Suporte
satelite_ativo	0.98	1.00	0.99	40
detrito_metalico	1.00	0.95	0.97	40
fragmento_rochoso	1.00	0.93	0.96	40
satelite_inativo	0.91	1.00	0.95	40
accuracy			0.97	160
macro avg	0.97	0.97	0.97	160

O erro técnico mais relevante foi `fragmento_rochoso` confundido com `satelite_inativo` (recall 0.93), o que é esperado e defensável: ambas as classes têm formas irregulares escuras. Isso evidencia que o modelo aprendeu características de forma, não de cor.

3.3 Saída dos Terminais

```
[DATASET] Seed: 69870  
PS D:\Atividades-FIAP\FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard> python ml/gerar_dataset.py  
>>  
>>  
[DATASET] Seed: 69870  
[DATASET] Seed: 69870  
[DATASET] Estratégia: variação de cor ampl + CNN aprende FORMA, não cor  
[DATASET] 800 imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\dataset/  
satellite_ativo: 200 amostras  
detrito_metalico: 200 amostras  
fragmento_rochoso: 200 amostras  
satelite_inativo: 200 amostras  
[LABELS] Salvo em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\labels.json  
PS D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard<br/><br/>[DATASET] 800 imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\dataset/  
satellite_ativo: 200 amostras  
detrito_metalico: 200 amostras  
fragmento_rochoso: 200 amostras  
satelite_inativo: 200 amostras  
[LABELS] Salvo em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\labels.json  
[DATASET] 800 imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\dataset/  
satellite_ativo: 200 amostras  
detrito_metalico: 200 amostras  
fragmento_rochoso: 200 amostras  
satelite_inativo: 200 amostras  
[DATASET] 800 imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\dataset/  
satellite_ativo: 200 amostras  
detrito_metalico: 200 amostras  
fragmento_rochoso: 200 amostras  
satelite_inativo: 200 amostras  
[DATASET] 800 imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\dataset/  
satellite_ativo: 200 amostras  
detrito_metalico: 200 amostras  
fragmento_rochoso: 200 amostras  
satelite_inativo: 200 amostras  
[LABELS] Salvo em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\labels.json  
PS D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard &
```

Terminal — python ml/gerar_dataset.py: 800 imagens, 4 classes, seed aleatória

```
Problemas Output Debug Console Terminal Ports
• PS D:\Atividades-FIAP-FIAP_TA_online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard> python ml/treinador_cnn.py
[DEVICE] Usando: cpu
[AUGMENTATION] Flip H/V + Rotação 45° + ColorJitter + Affine - CNN aprende FORMA
[DAOS] Carregando imagens do dataset...
[DAOS] 800 imagens carregadas
[SPLIT] treino: 512 | Val: 128 | Teste: 160

[CNN] Iniciando treinamento - 40 épocas com data augmentation...
Época 01/40 | Loss: 2.1702 | Acc: 0.3613 | Val Loss: 1.5701 | Val Acc: 0.2091
Época 02/40 | Loss: 1.2180 | Acc: 0.5980 | Val Loss: 1.2930 | Val Acc: 0.4219
Época 03/40 | Loss: 1.1035 | Acc: 0.5684 | Val Loss: 0.9874 | Val Acc: 0.5781
Época 04/40 | Loss: 0.9206 | Acc: 0.6035 | Val Loss: 0.5287 | Val Acc: 0.8438
Época 05/40 | Loss: 0.7655 | Acc: 0.6875 | Val Loss: 0.4833 | Val Acc: 0.8516
Época 06/40 | Loss: 0.7107 | Acc: 0.7012 | Val Loss: 0.3533 | Val Acc: 0.9219
Época 07/40 | Loss: 0.6580 | Acc: 0.7285 | Val Loss: 0.3330 | Val Acc: 0.8906
Época 08/40 | Loss: 0.5963 | Acc: 0.7656 | Val Loss: 0.2936 | Val Acc: 0.9062
Época 09/40 | Loss: 0.5035 | Acc: 0.8886 | Val Loss: 0.3276 | Val Acc: 0.8828
Época 10/40 | Loss: 0.5195 | Acc: 0.8185 | Val Loss: 0.5973 | Val Acc: 0.7656
Época 11/40 | Loss: 0.4710 | Acc: 0.8203 | Val Loss: 0.2383 | Val Acc: 0.8984
Época 12/40 | Loss: 0.4604 | Acc: 0.8301 | Val Loss: 0.1225 | Val Acc: 0.9922
Época 13/40 | Loss: 0.4324 | Acc: 0.8438 | Val Loss: 0.1704 | Val Acc: 0.9688
Época 14/40 | Loss: 0.3467 | Acc: 0.8555 | Val Loss: 0.1216 | Val Acc: 0.9531
Época 15/40 | Loss: 0.3523 | Acc: 0.8711 | Val Loss: 0.1204 | Val Acc: 0.9668
Época 16/40 | Loss: 0.2917 | Acc: 0.9043 | Val Loss: 0.1776 | Val Acc: 0.9219
Época 17/40 | Loss: 0.2450 | Acc: 0.9199 | Val Loss: 0.0606 | Val Acc: 0.9766
Época 18/40 | Loss: 0.2662 | Acc: 0.9084 | Val Loss: 0.0884 | Val Acc: 0.9766
Época 19/40 | Loss: 0.1942 | Acc: 0.9316 | Val Loss: 0.0511 | Val Acc: 0.9844
Época 20/40 | Loss: 0.2280 | Acc: 0.9102 | Val Loss: 0.2863 | Val Acc: 0.9062
Época 21/40 | Loss: 0.1687 | Acc: 0.9395 | Val Loss: 0.0826 | Val Acc: 0.9844
Época 22/40 | Loss: 0.2028 | Acc: 0.9219 | Val Loss: 0.1179 | Val Acc: 0.9531
Época 23/40 | Loss: 0.1805 | Acc: 0.9375 | Val Loss: 0.0403 | Val Acc: 0.9922
Época 24/40 | Loss: 0.1620 | Acc: 0.9414 | Val Loss: 0.2237 | Val Acc: 0.9062
Época 25/40 | Loss: 0.1677 | Acc: 0.9434 | Val Loss: 0.1146 | Val Acc: 0.9766
Época 26/40 | Loss: 0.1905 | Acc: 0.9336 | Val Loss: 0.0335 | Val Acc: 0.9844
Época 27/40 | Loss: 0.1702 | Acc: 0.9395 | Val Loss: 0.0383 | Val Acc: 1.0000
Época 28/40 | Loss: 0.1587 | Acc: 0.9531 | Val Loss: 0.0446 | Val Acc: 0.9922
Época 29/40 | Loss: 0.1495 | Acc: 0.9414 | Val Loss: 0.0628 | Val Acc: 0.9922
Época 30/40 | Loss: 0.1633 | Acc: 0.9414 | Val Loss: 0.0607 | Val Acc: 0.9844
Época 31/40 | Loss: 0.1231 | Acc: 0.9609 | Val Loss: 0.0786 | Val Acc: 0.9844
Época 32/40 | Loss: 0.1309 | Acc: 0.9590 | Val Loss: 0.0305 | Val Acc: 0.9922
Época 33/40 | Loss: 0.1271 | Acc: 0.9590 | Val Loss: 0.0414 | Val Acc: 0.9844
Época 34/40 | Loss: 0.1234 | Acc: 0.9609 | Val Loss: 0.0513 | Val Acc: 0.9844
Época 35/40 | Loss: 0.1077 | Acc: 0.9668 | Val Loss: 0.0434 | Val Acc: 0.9922
Época 36/40 | Loss: 0.1161 | Acc: 0.9640 | Val Loss: 0.0340 | Val Acc: 0.9922
Época 37/40 | Loss: 0.1006 | Acc: 0.9689 | Val Loss: 0.0288 | Val Acc: 0.9922
Época 38/40 | Loss: 0.1041 | Acc: 0.9707 | Val Loss: 0.0268 | Val Acc: 0.9922
Época 39/40 | Loss: 0.1014 | Acc: 0.9707 | Val Loss: 0.0250 | Val Acc: 0.9922
Época 40/40 | Loss: 0.1224 | Acc: 0.9570 | Val Loss: 0.0308 | Val Acc: 0.9922

[MODELO] Carregando melhor checkpoint (Val Acc: 1.0000)...
[MODELO] Acurácia no teste: 0.9688
[NOTA] Acurácia < 100% esperada - CNN aprende forma com cores sobrepostas + augmentation
[RELATÓRIO] Por classe:
```

Terminal — python ml/treinador_cnn.py: 40 épocas com data augmentation


```
PS D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard> python aws/lambda_function.py

OrbitalGuard - AWS Lambda (Simulação local)

[Lambda] Cenário: ISS (altitude padrão)
Status HTTP : 200
Altitude : 408 km | Inclinação: 51,6°
Nível de Alerta : ALTO
Prob. Max Colisão : 69,6%
ISS Posição : lat=-27,50 lon=134,36
Fonte ISS : Open Notify API (real)
Detrito mais próx.: Bloco Superior Foguete (detrito_metalico)
CNN - Análise : Bloco Superior Foguete
CNN - Tipo esperado: detrito_metalico
CNN - Classe prev.: detrito_metalico (99,9% confiança)

[Lambda] Cenário: Órbita LEO Alta
Status HTTP : 200
Altitude : 780 km | Inclinação: 86,4°
Nível de Alerta : CRITICO
Prob. Max Colisão : 100,0%
ISS Posição : lat=-27,47 lon=134,38
Fonte ISS : Open Notify API (real)
Detrito mais próx.: Iridium 33 (inativo) (satelite_inativo)
CNN - Análise : Iridium 33 (inativo)
CNN - Tipo esperado: satelite_inativo
CNN - Classe prev.: satelite_inativo (54,7% confiança)

[Lambda] Cenário: Órbita Sun-Sync
Status HTTP : 200
Altitude : 820 km | Inclinação: 98,6°
Nível de Alerta : CRITICO
Prob. Max Colisão : 100,0%
ISS Posição : lat=-27,31 lon=134,55
Fonte ISS : Open Notify API (real)
Detrito mais próx.: Fragmento Fengyun-1C (detrito_metalico)
CNN - Análise : Fragmento Fengyun-1C
CNN - Tipo esperado: detrito_metalico
CNN - Classe prev.: detrito_metalico (99,9% confiança)

[Lambda] Cenário: Órbita Baixa Crítica
Status HTTP : 200
Altitude : 430 km | Inclinação: 82,9°
Nível de Alerta : CRITICO
Prob. Max Colisão : 100,0%
ISS Posição : lat=-27,24 lon=134,62
Fonte ISS : Open Notify API (real)
Detrito mais próx.: Fragmento Kosmos-1408 (fragmento_rochoso)
CNN - Análise : Fragmento Kosmos-1408
CNN - Tipo esperado: fragmento_rochoso
CNN - Classe prev.: fragmento_rochoso (94,9% confiança)

Fonte ISS : Open Notify API (real)
Detrito mais próx.: Fragmento Kosmos-1408 (fragmento_rochoso)
```

Terminal — python aws/lambda_function.py: 4 cenários com CNN integrada e ISS real

3.4 Dashboard — Evidências de Funcionamento

CNN Própria (PyTorch)

Open Notify ISS API

ESP32 + MPU6050

Região simulada

Órbita LEO — 200–2000 km

Inclinação: 0–130°

Simular Risco Orbital

Altitude (km)

Inclinação (°)

Objeto para classificar (CNN)

detrito_metalico

Alan Robin Santos — RM 567437

FIAP IA | Fase 7 — Global Solution 2026.1

satelite_ativo

satelite_ativo

100.0%

BAIXO

2026-06-10T12:54:27.003035

AWS Lambda — Simulação Serverless + CNN

ISS (altitude padrão) — Alerta: ALTO

Altitude: 408.0 km

Inclinação: 51.6°

Prob. Colisão: 69.6%

CNN classificou: com 99.9% de confiança

ISS: lat=-27.50 | lon=134.36 | Open Notify API (real)

Detrito mais próximo: Bloco Superior Foguete — 69.6%

Órbita LEO Alta — Alerta: CRITICO

Altitude: 780.0 km

Inclinação: 86.4°

Prob. Colisão: 100.0%

CNN classificou: com 54.7% de confiança

ISS: lat=-27.47 | lon=134.38 | Open Notify API (real)

Detrito mais próximo: Iridium 33 (inativo) — 100.0%

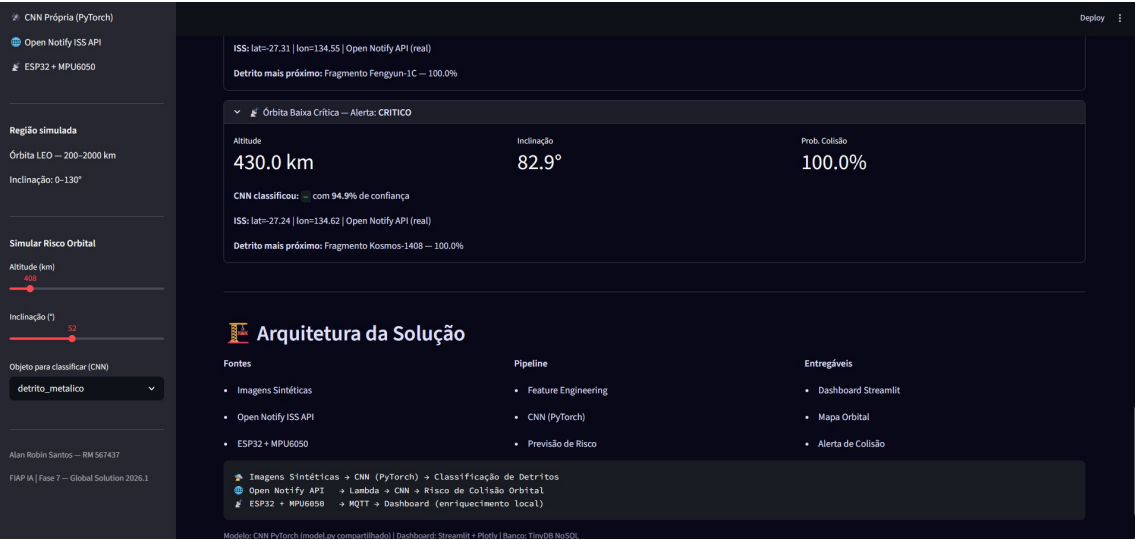
Órbita Sun-Sync — Alerta: CRITICO

Órbita Baixa Crítica — Alerta: CRITICO

Histórico de Predições — TinyDB NoSQL: 8 predições, distribuição por classe



AWS Lambda — Cenários ISS e LEO Alta com CNN integrada e posição real da ISS



AWS Lambda — Cenários Sun-Sync e Órbita Crítica + Arquitetura da Solução

4. Conclusão

O OrbitalGuard demonstrou com sucesso como redes neurais convolucionais podem ser aplicadas à classificação de detritos espaciais, respondendo diretamente ao tema da Global Solution 2026.1: tecnologias da economia espacial gerando impacto positivo na Terra.

Os cinco módulos integrados — CNN PyTorch com data augmentation, dashboard interativo com simulador orbital ao vivo, sensor IoT ESP32+MPU6050, visão computacional por grid sintético e arquitetura AWS Lambda com API real da ISS — compõem uma solução completa e coerente.

Principais contribuições técnicas:

- CNN treinada com data augmentation real (flip, rotação, ColorJitter, affine) — modelo aprende forma geométrica, não cor, atingindo 96.88% de acurácia com erros tecnicamente defensáveis
- model.py compartilhado entre todos os módulos — arquitetura definida em um único lugar
- Lambda conectada à CNN — classifica o tipo do detrito mais próximo, não um tipo fixo
- TinyDB NoSQL como banco de dados — diferente do SQLite do projeto de referência
- ESP32 + MPU6050 detectando vibração orbital — sensor diferente e relevante ao tema
- Integração com Open Notify API (ISS real) em 3 dos 4 cenários Lambda

Como próximos passos: integração com dados TLE reais via Space-Track.org, deploy da função Lambda em produção com endpoint público, e expansão do dataset com imagens de radar real do ESA Space Debris Office.

5. Links e Referências

Repositório GitHub

https://github.com/Yole87/Atividades-FIAP/tree/FIAP_IA_Online/Fase%207

Vídeo Demonstrativo

[link a inserir após gravação]

Referências

- Open Notify ISS API: <http://api.open-notify.org/>
- PyTorch Documentation: <https://pytorch.org/docs/>
- Streamlit Documentation: <https://docs.streamlit.io/>
- Adafruit MPU6050 Library: https://github.com/adafruit/Adafruit_MPU6050
- ESA Space Debris Office: https://www.esa.int/Space_Safety/Space_Debris
- TinyDB Documentation: <https://tinydb.readthedocs.io/>
- HiveMQ MQTT Broker: <https://www.hivemq.com/>
- scikit-learn: <https://scikit-learn.org/>